

PYTHON • DATA SCIENCE

Introduction to

NumPy

Numerical Python — From Basics to Practical Use

Hour 1

Introduction

Hours 2•3•4

Arrays & Operations





What is NumPy?



Definition

NumPy stands for Numerical Python. It is a powerful library that lets you work with large collections of numbers very efficiently — much faster than regular Python lists.



Why It Exists

Python lists are slow and limited for math operations. NumPy was built specifically to overcome this and brings the power of scientific computing into Python.



Core Object

The heart of NumPy is the ndarray (N-dimensional array) — a grid of values, all the same type. Think of it like a super-powered Excel column or table.

Why is NumPy Important?

50×

Faster than Python lists for math operations

N-D

Works in any dimension — 1D, 2D, 3D and beyond

1000+

Scientific libraries built on top of NumPy



Where NumPy is Used in Daily Life

Analyzing student exam scores

Processing weather temperature data

Medical image scanning (MRI, CT)

Stock market price calculations

Game physics & graphics rendering

Training AI & ML models

NumPy vs Python List — Key Differences

Feature	Python List	NumPy Array
Speed	Slow for math	Very Fast ✓
Data Types	Mixed allowed	One type only
Memory Usage	High usage	Optimized ✓
Math Ops	Manual loops needed	Direct operations ✓
Dimensions	1D only easily	1D, 2D, 3D+ ✓
Broadcasting	Not supported	Fully supported ✓

Real-Life Analogy — Think of It This Way



Spreadsheet Row (1D)

A 1D NumPy array is like a single row in Excel — a neat list of numbers side by side.
Example: [85, 90, 78, 95, 88] — scores of 5 students.



Spreadsheet Table (2D)

A 2D array is like a full spreadsheet — rows and columns.
Think of a student marks table with subjects as columns and students as rows.



Stacked Tables (3D)

A 3D array is like multiple spreadsheets stacked together.
Used in images: height × width × color channels (RGB).

Hours 2 · 3 · 4

Creating NumPy Arrays

From Python List · Built-in Methods · Random Functions



Method 1 — Creating Array from a Python List

What Does This Mean?

You already know Python lists like `[1, 2, 3]`. NumPy can take any Python list and convert it into a NumPy array using `np.array()`. The result looks similar but is stored differently in memory and works much more powerfully for math.

1D Array — Single Row of Numbers

Imagine scores of 5 students: `[85, 90, 78, 95, 88]`
This becomes a 1D array — a single straight line of values, just like one row in Excel.

2D Array — Rows and Columns

Example: `[[90, 85, 78], [88, 92, 80], [70, 75, 95]]`
Nested list → 3 rows, 3 cols. Just like a table.

How It Works — Step by Step

- 1: Start with a normal Python list, e.g. `[10, 20, 30]`
- 2: Pass it into `np.array()` function
- 3: NumPy reads each value and stores them in a contiguous memory block
- 4: All values must be same type (int or float) — mixed types auto-convert
- 5: Nested lists automatically become 2D arrays

Method 2 — Creating Arrays with Built-in Functions

`np.zeros()`

Creates an array filled entirely with 0s.

Use case: Initialize a results table where all values start as 0.

```
np.zeros(5) → [0.0.0.0.0.]
```

`np.ones()`

Creates an array filled entirely with 1s.

Use case: Start all weights in a calculation as 1.

```
np.ones((2,3)) → 2 rows, 3 cols of 1s
```

`np.arange()`

Creates evenly spaced numbers in a range — like Python's `range()` but returns a NumPy array.

Use case: Generate roll numbers 1 to 50.

```
np.arange(1,6) → [1 2 3 4 5]
```

`np.linspace()`

Creates exactly N evenly spaced values between start and stop.

Use case: Split temperature range 0°–100° into 10 equal points.

```
np.linspace(0,100,10) → 10 equal steps
```

`np.eye()`

Creates an Identity Matrix — 1s on the diagonal, 0s elsewhere.

Use case: Used in linear algebra and transformation equations.

```
np.eye(3) → 3x3 identity matrix
```

`np.full()`

Creates an array of any size filled with a specific value you choose.

Use case: Fill a placeholder array entirely with the number 99.

```
np.full((2,2), 7) → all 7s
```



Method 3 — Creating Arrays with Random Values

Random arrays are used for sample data, simulations, and testing. NumPy provides the `np.random` module for this.

`np.random.rand()`

Generates random decimal numbers between 0 and 1.
Real-life: Simulate random probability values — like rolling a biased die.

```
rand(3) → [0.42, 0.87, 0.12]
```

`np.random.randint()`

Generates random whole numbers between a given range.
Real-life: Simulate random exam scores between 40 and 100.

```
randint(1, 10, 5) → [3, 7, 2, 9, 4]
```

`np.random.randn()`

Generates numbers following a normal distribution (bell curve).
Real-life: Simulate heights and weights — values that cluster around an average.

```
randn(4) → [-0.5, 1.2, 0.3, -1.1]
```

`np.random.seed()`

Sets a starting point so random results are reproducible.
Real-life: Ensures test data looks the same every time you run the program.

```
seed(42) → fixed results always
```

Hour 3

Array Attributes & Methods

shape · dtype · size · ndim · reshape · max · min · argmax · argmin



Array Attributes — Inspecting Your Array

shape

Returns a tuple showing the structure of the array as (rows, columns).

Example: A 3×4 array → shape is (3, 4).

Think of it as: how many rows and columns does my table have?

dtype

Returns the data type of all elements in the array.

Example: int64 for whole numbers, float64 for decimals.

Think of it as: are the values in my column numbers or decimals?

size

Returns the total count of all elements in the array.

Example: A 3×4 array → size = 12 (3×4 = 12).

Think of it as: how many total cells fill my spreadsheet?

ndim

Returns the number of dimensions.

1D list = 1, 2D table = 2, 3D cube = 3.

Think of it as: is this a list, a table, or a cube of data?



Key Methods — reshape, max, min, argmax, argmin

`.reshape()`

Changes the shape of an array WITHOUT changing its data.

Imagine 12 students in 1 row → rearrange into 3 rows of 4 students each.

Total count stays the same!



Rule: rows × cols must equal total elements

`max()` / `min()`

Returns the highest / lowest value in the entire array.

Real-life: Find the top scorer and the lowest scorer in an exam.

Works on the full array or along rows/columns using axis.



axis=0 → column-wise | axis=1 → row-wise

`argmax()` / `argmin()`

Returns the INDEX (position) of the max or min value — NOT the value itself.

Real-life: 'Which student position got the highest marks?'

Crucial for finding WHO or WHERE, not just WHAT.



Position starts from 0 (zero-indexed)



Common Mistakes Students Make



✗ Reshape with wrong size



`np.zeros(12).reshape(5,3) → ERROR!`

Verify that `rows×cols` equals total elements. $5 \times 3 = 15 \neq 12$, so this will crash.



✗ Mixed types in array



`np.array([1, 2, 'hello']) → NumPy silently converts ALL values to strings. Your numbers become text!`



✗ `argmax` vs `max` confusion



`arr.max()` gives the VALUE e.g. 95

`arr.argmax()` gives the POSITION e.g. index 2.

Don't mix them up — they answer different questions.



✗ `shape` vs `size` confusion



`shape` → tuple (3,4) tells the structure.

`size` → single number 12 tells total count.

They answer different questions.

Hour 4

Operations on Arrays

Copying · Append & Insert · Sorting · Deleting · Combining · Splitting





Copying Arrays — `copy()` vs `view()`



`.copy()` — Deep Copy (Safe)

Creates a completely independent duplicate of the original array.

If you change the copy → the original is NOT affected.

Real-life analogy: Like photocopying a document. You can write on the photocopy without touching the original paper.

When to use: Whenever you want to experiment with data without risking the original values.



`.view()` — Shallow Copy (Linked)

Creates a reference (window) into the SAME data — not a new copy.

If you change the view → the ORIGINAL changes too!

Real-life analogy: Looking at the original document through a glass window. If someone edits the paper, you see the change.

When to use: When you need a slice of data and intentionally want changes to reflect back.



Adding Elements — `append()` and `insert()`

`np.append()` — Add to the End

Adds one or more elements at the END of an array.

Important: NumPy `append` does NOT modify the original — it returns a NEW array.

Real-life: A teacher adding a new student's marks at the end of the class list.

Note: The result is always flattened to 1D unless you specify an axis parameter.

`np.insert()` — Add at Specific Position

Inserts an element at a SPECIFIC position (index) that you choose.

Syntax: `np.insert(array, index, value)`

Real-life: Adding a late-registered student's name at position 3 in a sorted class list.

Index 0 = first position. Index 2 = third position from start.

Like `append()`, it returns a NEW array — does not change the original.

⚠ Key Difference: `append()` always adds at END. `insert()` lets you choose EXACTLY WHERE to add.

Sorting & Removing Elements



np.sort() — Sorting Arrays

- Returns a SORTED COPY of the array — original stays unchanged
- Default order: ascending (smallest → largest)
- For descending order: sort then reverse with `[::-1]`
- Works on 2D arrays too — sort per row or per column using axis
- Real-life: Sorting student scores from lowest to highest to create rankings



np.delete() — Removing Elements

- Removes element(s) at a given index or set of indices
- Syntax: `np.delete(array, index)`
- Does NOT modify the original — returns a NEW array
- Delete multiple at once: `np.delete(arr, [0, 2])`
- Real-life: Remove an absent student's score before calculating class average

Combining & Splitting Arrays



Combining — concatenate & stack

`np.concatenate()`

Joins two or more arrays end-to-end.

Real-life: Merge marks of Class A and Class B into one list.

`np.vstack()`

Stacks arrays vertically (adds rows).

Real-life: Adding a new student's row beneath existing rows in a table.

`np.hstack()`

Stacks arrays horizontally (adds columns).

Real-life: Adding a new subject column to an existing marks table.



Splitting — split & array_split

`np.split()`

Splits into EQUAL parts — raises error if unequal.

Real-life: Divide 12 students into 3 equal groups of 4.

`np.array_split()`

Splits into parts — allows UNEQUAL sizes, no error.

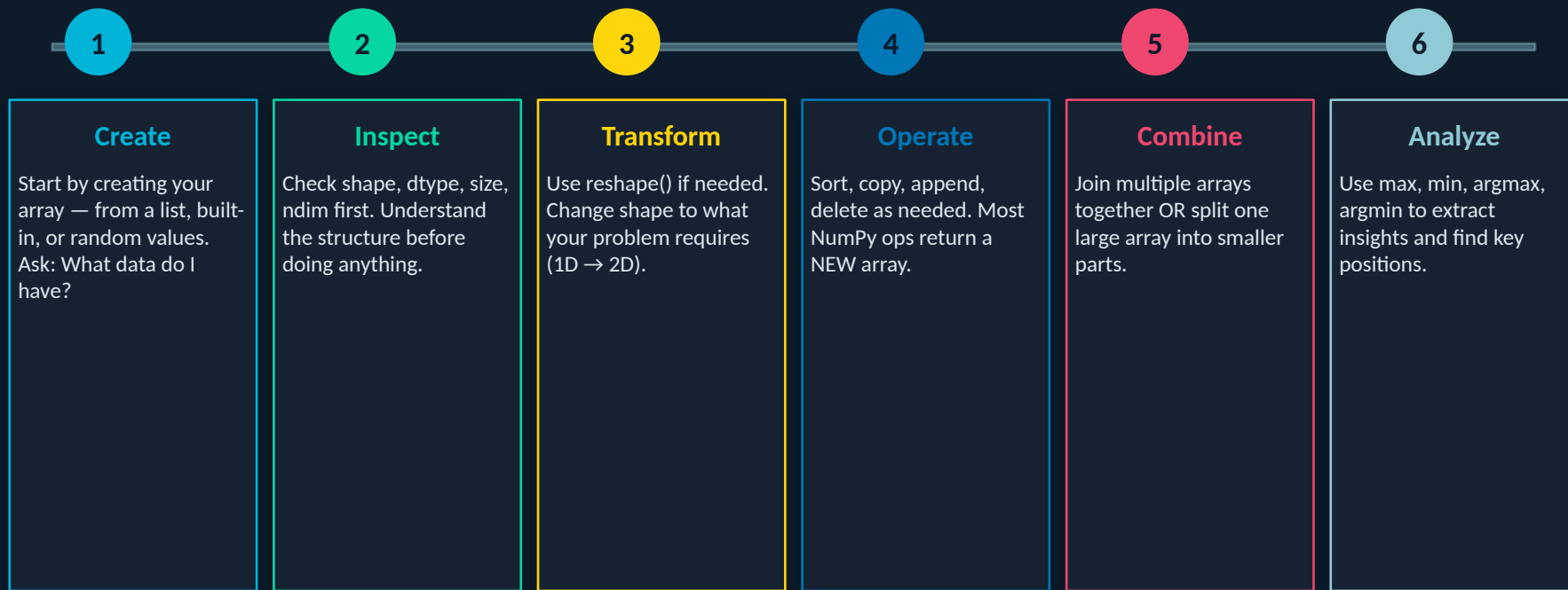
Real-life: Divide 10 students into 3 groups (4, 3, 3).

`np.hsplit()` / `np.vsplit()`

Splits horizontally (columns) or vertically (rows).

Real-life: Split a grade table by subjects or by student groups.

How to Think When Working with NumPy Arrays





How to Think While Solving NumPy Problems

Before you write code, ask:

- What shape is my data? (1D list? 2D table?)
- What operation do I need? (sorting? filtering? finding max?)
- Will I need the original later? → Always use `copy()` to be safe

While working with methods:

- Check `.shape` BEFORE reshape — total count must match
- Use `.dtype` to know if values are ints or floats
- `argmax` gives position, `max` gives value — decide which you need

When combining or splitting:

- Use `concatenate` for flat joining, `vstack/hstack` for 2D tables
- Use `array_split` instead of `split` to avoid errors on uneven sizes
- Always print `.shape` after combine/split to verify the result

Summary — Key Takeaways

Hour 1 • NumPy Basics

- NumPy = fast, efficient numerical computing in Python
- ndarray is the core object — grid of same-type values
- 50× faster than Python lists for math operations

Hours 2-3 • Creating Arrays

- From lists: `np.array()` — 1D and 2D
- Built-in: `zeros`, `ones`, `arange`, `linspace`, `eye`, `full`
- Random: `rand`, `randint`, `randn`, `seed`

Hour 3 • Attributes & Methods

- Inspect: `shape`, `dtype`, `size`, `ndim`
- Transform: `reshape()` — rows×cols must equal total
- Analyze: `max`, `min`, `argmax`, `argmin`

Hour 4 • Operations

- Copy carefully: `copy()` vs `view()`
- Add: `append (end)` | `insert (any position)`
- Sort, delete, concatenate, stack, split

NumPy is the foundation of all Data Science in Python — master it and everything else becomes easier!